

FFNN Hyperparameter Grid Version

August 29, 2025

```
[1]: # This is an example of the code used to grid search through the FFNN written
      ↪ by Barrett Viator...
      # it takes a very long time to execute the computations and is here as an
      ↪ example but not used in final code
```

```
[2]: # Import necessary libraries
import pandas as pd
import numpy as np
from scipy.sparse import csr_matrix, hstack
from sklearn.preprocessing import OneHotEncoder, MinMaxScaler
from sklearn.model_selection import train_test_split
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LeakyReLU
from tensorflow.keras.layers import ReLU
from tensorflow.keras.optimizers import Adam, RMSprop, SGD
from tensorflow.keras.callbacks import EarlyStopping# Added for tuning
import matplotlib.pyplot as plt
import pyhere
```

```
[3]: train_path = pyhere.here('Data', 'train_data.xlsx')
test_path = pyhere.here('Data', 'test_data.xlsx')
train_data = pd.read_excel(train_path)
test_data = pd.read_excel(test_path)
# Use dropna() to remove any rows with missing values
train_data = train_data.dropna()
test_data = test_data.dropna()
print(train_data.head())
```

	CountryName	CountryCode	year	HDI_Index	\
0	Antigua and Barbuda	ATG	2022	0.848	
9	Antigua and Barbuda	ATG	2008	0.830	
10	Antigua and Barbuda	ATG	2009	0.827	
11	Antigua and Barbuda	ATG	2010	0.828	
12	Antigua and Barbuda	ATG	2011	0.828	

	HDI_Category	FoodIndex	ElectricAccess	PopDensity	\
--	--------------	-----------	----------------	------------	---

0	Very high human development	89.95	100.0	211.000000
9	Very high human development	145.98	100.0	188.436364
10	Very high human development	141.40	98.5	191.302273
11	Very high human development	102.49	98.7	193.909091
12	Very high human development	104.99	94.6	196.209091

	PopInSlums	WaterStress	...	IncomeGroup	YearlyChgInternet	\
0	2.646340	8.461538	...	High income	0.997844	
9	2.646317	9.230769	...	High income	3.164091	
10	2.646311	9.038462	...	High income	3.164091	
11	2.646300	8.846154	...	High income	3.831893	
12	2.646300	8.653846	...	High income	3.831893	

	YearlyChgHDI	Lag1_InternetUsersPct	Lag2_InternetUsersPct	\
0	0.005	8.247411	7.147796	
9	0.003	5.957120	5.079504	
10	-0.003	6.242918	5.336653	
11	0.001	6.508026	5.571669	
12	0.000	6.815164	5.788459	

	Lag1_YearlyChgInternet	Lag2_YearlyChgInternet	\
0	0.993070	0.908060	
9	3.128215	2.432146	
10	3.128215	3.116179	
11	3.128215	3.116179	
12	3.782704	3.116179	

	Cumulative3yrChg_InternetUsersPct	GovernanceScore	HealthSpend
0	3.2	0.180504	853.834335
9	11.0	0.904398	523.176632
10	12.0	0.929469	472.044125
11	13.0	0.914123	573.175838
12	14.0	0.898586	562.880747

[5 rows x 30 columns]

```
[4]: train_target = train_data['HDI_Index']
test_target = test_data['HDI_Index']

train_predictors = train_data.drop('HDI_Index', axis=1)
test_predictors = test_data.drop('HDI_Index', axis=1)

train_predictors = train_data[['CountryCode', 'FoodIndex', 'ElectricAccess',
↪ 'PopDensity', 'PopInSlums', 'WaterStress', 'InternetUsersPct',
                                'GDPPerCapGrowth', 'GDPGrowth', 'GDP',
↪ 'PoliticalStability', 'RqdEduYears', 'GovtEduSpendPctGDP',
```

```

        'UHCSERVICECoverage', 'RuralPopulGrowth',
        ↪ 'VoiceAccountability']]

test_predictors = test_data[['CountryCode', 'FoodIndex', 'ElectricAccess',
        ↪ 'PopDensity', 'PopInSlums', 'WaterStress', 'InternetUsersPct',
        'GDPPERCapGrowth', 'GDPGrowth', 'GDP',
        ↪ 'PoliticalStability', 'RqdEduYears', 'GovtEduSpendPctGDP',
        'UHCSERVICECoverage', 'RuralPopulGrowth',
        ↪ 'VoiceAccountability']]

```

```

[5]: # Combine the predictor data frames to ensure consistent encoding
combined_predictors = pd.concat([train_predictors, test_predictors],
        ↪ ignore_index=True)

# Identify numerical and categorical columns
numerical_cols = combined_predictors.select_dtypes(include=np.number).columns
categorical_cols = combined_predictors.select_dtypes(include='object').columns

# Initialize MinMaxScaler
scaler = MinMaxScaler()

# Fit the scaler on the combined numerical data to learn the min/max values
combined_predictors[numerical_cols] = scaler.
        ↪ fit_transform(combined_predictors[numerical_cols])

# Initialize OneHotEncoder to create sparse output and handle unknown categories
ohe = OneHotEncoder(sparse_output=False, handle_unknown='ignore')

# Fit the encoder on the combined data to learn all possible categories
combined_categorical_encoded = ohe.
        ↪ fit_transform(combined_predictors[categorical_cols])

# Combine the encoded categorical data with the numerical data
combined_features = np.concatenate([combined_predictors[numerical_cols].values,
        combined_categorical_encoded], axis=1)

# Split the combined feature matrix back into training and test sets
train_features = combined_features[:len(train_predictors)]
test_features = combined_features[len(train_predictors):]

# Features split
train_features, test_features, train_target, test_target = train_test_split(
    combined_features, train_target, test_size=0.2, random_state=42
)

```

```
[6]: # Hyperparameter tuning with grid search
hyper_grid = {
    'epochs': [1000, 2000],
    'batch_size': [16, 32, 64, 128],
    'learning_rate': [0.0001, 0.001, 0.01, 0.1]
}
results = []

[7]: # Hyperparameter tuning loop
for epochs in hyper_grid['epochs']:
    for batch_size in hyper_grid['batch_size']:
        for lr in hyper_grid['learning_rate']:
            print(f"\nTraining with epochs={epochs}, batch_size={batch_size},
↪lr={lr}")

            model = Sequential()
            model.add(Dense(64, input_dim=train_features.shape[1]))
            model.add(ReLU())
            model.add(Dense(32))
            model.add(ReLU())
            model.add(Dense(1, activation='linear'))

            optimizer = RMSprop(learning_rate=lr)
            model.compile(optimizer=optimizer, loss='mean_absolute_error',
↪metrics=['mae'])

            early_stop = EarlyStopping(monitor='val_loss', patience=50,
↪verbose=0, restore_best_weights=True)

            history = model.fit(
                train_features, train_target,
                epochs=epochs,
                batch_size=batch_size,
                validation_split=0.2,
                callbacks=[early_stop],
                verbose=0
            )

            # Evaluate safely
            if test_features.shape[0] == 0 or test_target.shape[0] == 0:
                print("Skipping evaluation: Test data is empty.")
                loss, mae = None, None
            else:
                eval_results = model.evaluate(test_features, test_target,
↪verbose=0)

                if isinstance(eval_results, (list, tuple)):
```

```

        if len(eval_results) >= 2:
            loss, mae = eval_results[0], eval_results[1]
        elif len(eval_results) == 1:
            loss, mae = eval_results[0], None
        elif len(eval_results) == 0:
            print("evaluate() returned empty list. Setting loss/mae_
↪= None.")
            loss, mae = None, None
        else:
            loss, mae = eval_results, None

    results.append({
        'epochs': epochs,
        'batch_size': batch_size,
        'learning_rate': lr,
        'loss': loss,
        'mae': mae
    })

# Put results in DataFrame
results_df = pd.DataFrame(results)
results_df = results_df.dropna(subset=['loss'])

if not results_df.empty:
    print("\nAll Hyperparameter Results:")
    print(results_df)

    best_params = results_df.loc[results_df['loss'].idxmin()]
    print("\nBest Hyperparameters Found:")
    print(best_params)

    # Train the Final Model with Best Parameters
    print("\nTraining final model with best parameters...")
    final_model = Sequential()
    final_model.add(Dense(256, input_dim=train_features.shape[1],
↪activation='relu'))
    final_model.add(Dense(128, activation='relu'))
    final_model.add(Dense(64, activation='relu'))
    final_model.add(Dense(1, activation='linear'))

    final_optimizer = RMSprop(learning_rate=best_params['learning_rate'])
    final_model.compile(optimizer=final_optimizer, loss='mean_absolute_error',
↪metrics=['mae'])

    early_stop_final = EarlyStopping(monitor='val_loss', patience=50,
↪verbose=1, restore_best_weights=True)

```

```

final_history = final_model.fit(
    train_features, train_target,
    epochs=1000,
    batch_size=int(best_params['batch_size']),
    validation_data=(test_features, test_target),
    callbacks=[early_stop_final],
    verbose=1
)

# Final Evaluation
if test_features.shape[0] == 0 or test_target.shape[0] == 0:
    print("Skipping final evaluation: Test data is empty.")
    loss_final, mae_final = None, None
else:
    eval_results_final = final_model.evaluate(test_features, test_target,
↪ verbose=0)

    if isinstance(eval_results_final, (list, tuple)):
        if len(eval_results_final) >= 2:
            loss_final, mae_final = eval_results_final[0],
↪ eval_results_final[1]
            elif len(eval_results_final) == 1:
                loss_final, mae_final = eval_results_final[0], None
            elif len(eval_results_final) == 0:
                print("evaluate() returned empty list for final model.")
                loss_final, mae_final = None, None
        else:
            loss_final, mae_final = eval_results_final, None

    if loss_final is not None:
        print(f"\nFinal Model Loss on Test Data: {loss_final:.4f}")
    if mae_final is not None:
        print(f"Final Model MAE on Test Data: {mae_final:.4f}")

# Predictions and Plots
predictions = final_model.predict(test_features)
if not np.any(np.isnan(predictions)):
    predictions = predictions.flatten()
    final_results_df = pd.DataFrame({'Actual': test_target, 'Predicted':
↪ predictions})

    # Scatter plot
    plt.figure(figsize=(10, 6))
    plt.scatter(final_results_df['Actual'],
↪ final_results_df['Predicted'], alpha=0.6)
    plt.plot([min(final_results_df['Actual']),
↪ max(final_results_df['Actual'])],

```

```

        [min(final_results_df['Actual']),
↪max(final_results_df['Actual'])],
        color='red', linestyle='--', lw=2)
    plt.title('Actual vs. Predicted Values')
    plt.xlabel('Actual Internet Users Percentage')
    plt.ylabel('Predicted Internet Users Percentage')
    plt.show()

    # Residual plot
    final_results_df['Residuals'] = final_results_df['Actual'] -
↪final_results_df['Predicted']
    plt.figure(figsize=(10, 6))
    plt.hist(final_results_df['Residuals'], bins=30, edgecolor='black',
↪alpha=0.7)
    plt.axvline(x=0, color='red', linestyle='--', lw=2)
    plt.title('Residuals Distribution')
    plt.xlabel('Residuals (Actual - Predicted)')
    plt.ylabel('Frequency')
    plt.show()

    # Training & validation MAE
    mae_history = final_history.history['mae']
    val_mae_history = final_history.history['val_mae']
    epochs_range = range(1, len(mae_history) + 1)
    plt.figure(figsize=(10, 6))
    plt.plot(epochs_range, mae_history, 'b', label='Training MAE')
    plt.plot(epochs_range, val_mae_history, 'r', label='Validation MAE')
    plt.title('Training and Validation MAE over Epochs')
    plt.xlabel('Epochs')
    plt.ylabel('MAE')
    plt.legend()
    plt.show()
    else:
        print("Predictions contain invalid values (NaN or infinity). Cannot
↪plot.")
    else:
        print("No valid results found. Cannot train final model.")

```

Training with epochs=1000, batch_size=16, lr=0.0001

C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
 UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When
 using Sequential models, prefer using an `Input(shape)` object as the first
 layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=1000, batch_size=16, lr=0.001

C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=1000, batch_size=16, lr=0.01

C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=1000, batch_size=16, lr=0.1

C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=1000, batch_size=32, lr=0.0001

C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=1000, batch_size=32, lr=0.001

C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=1000, batch_size=32, lr=0.01

C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```


Training with epochs=1000, batch_size=32, lr=0.1

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=1000, batch_size=64, lr=0.0001

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=1000, batch_size=64, lr=0.001

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=1000, batch_size=64, lr=0.01

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=1000, batch_size=64, lr=0.1

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=1000, batch_size=128, lr=0.0001

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=1000, batch_size=128, lr=0.001

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=1000, batch_size=128, lr=0.01

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=1000, batch_size=128, lr=0.1

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=2000, batch_size=16, lr=0.0001

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=2000, batch_size=16, lr=0.001

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=2000, batch_size=16, lr=0.01

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=2000, batch_size=16, lr=0.1

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=2000, batch_size=32, lr=0.0001

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=2000, batch_size=32, lr=0.001

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=2000, batch_size=32, lr=0.01

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=2000, batch_size=32, lr=0.1

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=2000, batch_size=64, lr=0.0001

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=2000, batch_size=64, lr=0.001

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=2000, batch_size=64, lr=0.01

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=2000, batch_size=64, lr=0.1

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=2000, batch_size=128, lr=0.0001

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=2000, batch_size=128, lr=0.001

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=2000, batch_size=128, lr=0.01

```
C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Training with epochs=2000, batch_size=128, lr=0.1

C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

All Hyperparameter Results:

	epochs	batch_size	learning_rate	loss	mae
0	1000	16	0.0001	0.006557	0.006557
1	1000	16	0.0010	0.006885	0.006885
2	1000	16	0.0100	0.009586	0.009586
3	1000	16	0.1000	0.130775	0.130775
4	1000	32	0.0001	0.006877	0.006877
5	1000	32	0.0010	0.006661	0.006661
6	1000	32	0.0100	0.011462	0.011462
7	1000	32	0.1000	0.130820	0.130820
8	1000	64	0.0001	0.006984	0.006984
9	1000	64	0.0010	0.007646	0.007646
10	1000	64	0.0100	0.013756	0.013756
11	1000	64	0.1000	0.073112	0.073112
12	1000	128	0.0001	0.008575	0.008575
13	1000	128	0.0010	0.012860	0.012860
14	1000	128	0.0100	0.011175	0.011175
15	1000	128	0.1000	0.094856	0.094856
16	2000	16	0.0001	0.006971	0.006971
17	2000	16	0.0010	0.005912	0.005912
18	2000	16	0.0100	0.011418	0.011418
19	2000	16	0.1000	0.130737	0.130737
20	2000	32	0.0001	0.007294	0.007294
21	2000	32	0.0010	0.007281	0.007281
22	2000	32	0.0100	0.013122	0.013122
23	2000	32	0.1000	0.110901	0.110901
24	2000	64	0.0001	0.007501	0.007501
25	2000	64	0.0010	0.008477	0.008477
26	2000	64	0.0100	0.011615	0.011615
27	2000	64	0.1000	0.071729	0.071729
28	2000	128	0.0001	0.007837	0.007837
29	2000	128	0.0010	0.011122	0.011122
30	2000	128	0.0100	0.016060	0.016060
31	2000	128	0.1000	0.073599	0.073599

Best Hyperparameters Found:

epochs	2000.000000
batch_size	16.000000
learning_rate	0.001000

```
loss          0.005912
mae           0.005912
Name: 17, dtype: float64
```

Training final model with best parameters...

Epoch 1/1000

C:\Users\barre\my_new_env2\Lib\site-packages\keras\src\layers\core\dense.py:92:
UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
114/114          1s 4ms/step -
loss: 0.0750 - mae: 0.0750 - val_loss: 0.0493 - val_mae: 0.0493
Epoch 2/1000
114/114          0s 3ms/step -
loss: 0.0446 - mae: 0.0446 - val_loss: 0.0676 - val_mae: 0.0676
Epoch 3/1000
114/114          0s 3ms/step -
loss: 0.0361 - mae: 0.0361 - val_loss: 0.0533 - val_mae: 0.0533
Epoch 4/1000
114/114          0s 3ms/step -
loss: 0.0336 - mae: 0.0336 - val_loss: 0.0345 - val_mae: 0.0345
Epoch 5/1000
114/114          0s 3ms/step -
loss: 0.0329 - mae: 0.0329 - val_loss: 0.0454 - val_mae: 0.0454
Epoch 6/1000
114/114          0s 3ms/step -
loss: 0.0301 - mae: 0.0301 - val_loss: 0.0194 - val_mae: 0.0194
Epoch 7/1000
114/114          0s 3ms/step -
loss: 0.0292 - mae: 0.0292 - val_loss: 0.0242 - val_mae: 0.0242
Epoch 8/1000
114/114          0s 3ms/step -
loss: 0.0283 - mae: 0.0283 - val_loss: 0.0334 - val_mae: 0.0334
Epoch 9/1000
114/114          0s 3ms/step -
loss: 0.0271 - mae: 0.0271 - val_loss: 0.0334 - val_mae: 0.0334
Epoch 10/1000
114/114          0s 3ms/step -
loss: 0.0258 - mae: 0.0258 - val_loss: 0.0156 - val_mae: 0.0156
Epoch 11/1000
114/114          0s 3ms/step -
loss: 0.0258 - mae: 0.0258 - val_loss: 0.0225 - val_mae: 0.0225
Epoch 12/1000
114/114          0s 3ms/step -
loss: 0.0240 - mae: 0.0240 - val_loss: 0.0324 - val_mae: 0.0324
Epoch 13/1000
```

```

114/114          0s 3ms/step -
loss: 0.0242 - mae: 0.0242 - val_loss: 0.0454 - val_mae: 0.0454
Epoch 14/1000
114/114          0s 3ms/step -
loss: 0.0235 - mae: 0.0235 - val_loss: 0.0129 - val_mae: 0.0129
Epoch 15/1000
114/114          0s 3ms/step -
loss: 0.0225 - mae: 0.0225 - val_loss: 0.0253 - val_mae: 0.0253
Epoch 16/1000
114/114          0s 3ms/step -
loss: 0.0219 - mae: 0.0219 - val_loss: 0.0242 - val_mae: 0.0242
Epoch 17/1000
114/114          0s 3ms/step -
loss: 0.0215 - mae: 0.0215 - val_loss: 0.0325 - val_mae: 0.0325
Epoch 18/1000
114/114          0s 3ms/step -
loss: 0.0214 - mae: 0.0214 - val_loss: 0.0242 - val_mae: 0.0242
Epoch 19/1000
114/114          0s 3ms/step -
loss: 0.0213 - mae: 0.0213 - val_loss: 0.0182 - val_mae: 0.0182
Epoch 20/1000
114/114          0s 3ms/step -
loss: 0.0205 - mae: 0.0205 - val_loss: 0.0309 - val_mae: 0.0309
Epoch 21/1000
114/114          0s 3ms/step -
loss: 0.0209 - mae: 0.0209 - val_loss: 0.0365 - val_mae: 0.0365
Epoch 22/1000
114/114          0s 3ms/step -
loss: 0.0207 - mae: 0.0207 - val_loss: 0.0292 - val_mae: 0.0292
Epoch 23/1000
114/114          0s 3ms/step -
loss: 0.0199 - mae: 0.0199 - val_loss: 0.0139 - val_mae: 0.0139
Epoch 24/1000
114/114          0s 3ms/step -
loss: 0.0196 - mae: 0.0196 - val_loss: 0.0216 - val_mae: 0.0216
Epoch 25/1000
114/114          0s 3ms/step -
loss: 0.0194 - mae: 0.0194 - val_loss: 0.0335 - val_mae: 0.0335
Epoch 26/1000
114/114          0s 3ms/step -
loss: 0.0194 - mae: 0.0194 - val_loss: 0.0157 - val_mae: 0.0157
Epoch 27/1000
114/114          0s 3ms/step -
loss: 0.0187 - mae: 0.0187 - val_loss: 0.0149 - val_mae: 0.0149
Epoch 28/1000
114/114          0s 3ms/step -
loss: 0.0182 - mae: 0.0182 - val_loss: 0.0177 - val_mae: 0.0177
Epoch 29/1000

```

```

114/114          0s 3ms/step -
loss: 0.0183 - mae: 0.0183 - val_loss: 0.0106 - val_mae: 0.0106
Epoch 30/1000
114/114          0s 3ms/step -
loss: 0.0176 - mae: 0.0176 - val_loss: 0.0112 - val_mae: 0.0112
Epoch 31/1000
114/114          0s 3ms/step -
loss: 0.0178 - mae: 0.0178 - val_loss: 0.0222 - val_mae: 0.0222
Epoch 32/1000
114/114          0s 3ms/step -
loss: 0.0169 - mae: 0.0169 - val_loss: 0.0098 - val_mae: 0.0098
Epoch 33/1000
114/114          0s 3ms/step -
loss: 0.0169 - mae: 0.0169 - val_loss: 0.0171 - val_mae: 0.0171
Epoch 34/1000
114/114          0s 3ms/step -
loss: 0.0172 - mae: 0.0172 - val_loss: 0.0152 - val_mae: 0.0152
Epoch 35/1000
114/114          0s 3ms/step -
loss: 0.0167 - mae: 0.0167 - val_loss: 0.0102 - val_mae: 0.0102
Epoch 36/1000
114/114          0s 3ms/step -
loss: 0.0166 - mae: 0.0166 - val_loss: 0.0280 - val_mae: 0.0280
Epoch 37/1000
114/114          0s 3ms/step -
loss: 0.0163 - mae: 0.0163 - val_loss: 0.0155 - val_mae: 0.0155
Epoch 38/1000
114/114          0s 3ms/step -
loss: 0.0158 - mae: 0.0158 - val_loss: 0.0189 - val_mae: 0.0189
Epoch 39/1000
114/114          0s 3ms/step -
loss: 0.0159 - mae: 0.0159 - val_loss: 0.0181 - val_mae: 0.0181
Epoch 40/1000
114/114          0s 3ms/step -
loss: 0.0152 - mae: 0.0152 - val_loss: 0.0248 - val_mae: 0.0248
Epoch 41/1000
114/114          0s 3ms/step -
loss: 0.0154 - mae: 0.0154 - val_loss: 0.0196 - val_mae: 0.0196
Epoch 42/1000
114/114          0s 3ms/step -
loss: 0.0149 - mae: 0.0149 - val_loss: 0.0175 - val_mae: 0.0175
Epoch 43/1000
114/114          0s 3ms/step -
loss: 0.0149 - mae: 0.0149 - val_loss: 0.0101 - val_mae: 0.0101
Epoch 44/1000
114/114          0s 3ms/step -
loss: 0.0147 - mae: 0.0147 - val_loss: 0.0196 - val_mae: 0.0196
Epoch 45/1000

```



```

114/114          0s 3ms/step -
loss: 0.0149 - mae: 0.0149 - val_loss: 0.0085 - val_mae: 0.0085
Epoch 46/1000
114/114          0s 3ms/step -
loss: 0.0147 - mae: 0.0147 - val_loss: 0.0090 - val_mae: 0.0090
Epoch 47/1000
114/114          0s 3ms/step -
loss: 0.0150 - mae: 0.0150 - val_loss: 0.0109 - val_mae: 0.0109
Epoch 48/1000
114/114          0s 3ms/step -
loss: 0.0140 - mae: 0.0140 - val_loss: 0.0222 - val_mae: 0.0222
Epoch 49/1000
114/114          0s 3ms/step -
loss: 0.0140 - mae: 0.0140 - val_loss: 0.0140 - val_mae: 0.0140
Epoch 50/1000
114/114          0s 3ms/step -
loss: 0.0141 - mae: 0.0141 - val_loss: 0.0166 - val_mae: 0.0166
Epoch 51/1000
114/114          0s 3ms/step -
loss: 0.0139 - mae: 0.0139 - val_loss: 0.0090 - val_mae: 0.0090
Epoch 52/1000
114/114          0s 3ms/step -
loss: 0.0141 - mae: 0.0141 - val_loss: 0.0188 - val_mae: 0.0188
Epoch 53/1000
114/114          0s 3ms/step -
loss: 0.0140 - mae: 0.0140 - val_loss: 0.0117 - val_mae: 0.0117
Epoch 54/1000
114/114          0s 3ms/step -
loss: 0.0134 - mae: 0.0134 - val_loss: 0.0150 - val_mae: 0.0150
Epoch 55/1000
114/114          0s 3ms/step -
loss: 0.0140 - mae: 0.0140 - val_loss: 0.0125 - val_mae: 0.0125
Epoch 56/1000
114/114          0s 3ms/step -
loss: 0.0136 - mae: 0.0136 - val_loss: 0.0138 - val_mae: 0.0138
Epoch 57/1000
114/114          0s 3ms/step -
loss: 0.0133 - mae: 0.0133 - val_loss: 0.0199 - val_mae: 0.0199
Epoch 58/1000
114/114          0s 3ms/step -
loss: 0.0135 - mae: 0.0135 - val_loss: 0.0108 - val_mae: 0.0108
Epoch 59/1000
114/114          0s 3ms/step -
loss: 0.0130 - mae: 0.0130 - val_loss: 0.0128 - val_mae: 0.0128
Epoch 60/1000
114/114          0s 3ms/step -
loss: 0.0132 - mae: 0.0132 - val_loss: 0.0084 - val_mae: 0.0084
Epoch 61/1000

```

```

114/114          0s 3ms/step -
loss: 0.0135 - mae: 0.0135 - val_loss: 0.0180 - val_mae: 0.0180
Epoch 62/1000
114/114          0s 3ms/step -
loss: 0.0128 - mae: 0.0128 - val_loss: 0.0167 - val_mae: 0.0167
Epoch 63/1000
114/114          0s 3ms/step -
loss: 0.0129 - mae: 0.0129 - val_loss: 0.0152 - val_mae: 0.0152
Epoch 64/1000
114/114          0s 3ms/step -
loss: 0.0130 - mae: 0.0130 - val_loss: 0.0139 - val_mae: 0.0139
Epoch 65/1000
114/114          0s 3ms/step -
loss: 0.0128 - mae: 0.0128 - val_loss: 0.0100 - val_mae: 0.0100
Epoch 66/1000
114/114          0s 3ms/step -
loss: 0.0125 - mae: 0.0125 - val_loss: 0.0091 - val_mae: 0.0091
Epoch 67/1000
114/114          0s 3ms/step -
loss: 0.0128 - mae: 0.0128 - val_loss: 0.0193 - val_mae: 0.0193
Epoch 68/1000
114/114          0s 3ms/step -
loss: 0.0126 - mae: 0.0126 - val_loss: 0.0085 - val_mae: 0.0085
Epoch 69/1000
114/114          0s 3ms/step -
loss: 0.0124 - mae: 0.0124 - val_loss: 0.0155 - val_mae: 0.0155
Epoch 70/1000
114/114          0s 3ms/step -
loss: 0.0122 - mae: 0.0122 - val_loss: 0.0239 - val_mae: 0.0239
Epoch 71/1000
114/114          0s 3ms/step -
loss: 0.0128 - mae: 0.0128 - val_loss: 0.0133 - val_mae: 0.0133
Epoch 72/1000
114/114          0s 3ms/step -
loss: 0.0126 - mae: 0.0126 - val_loss: 0.0097 - val_mae: 0.0097
Epoch 73/1000
114/114          0s 3ms/step -
loss: 0.0121 - mae: 0.0121 - val_loss: 0.0134 - val_mae: 0.0134
Epoch 74/1000
114/114          0s 3ms/step -
loss: 0.0123 - mae: 0.0123 - val_loss: 0.0079 - val_mae: 0.0079
Epoch 75/1000
114/114          0s 3ms/step -
loss: 0.0121 - mae: 0.0121 - val_loss: 0.0090 - val_mae: 0.0090
Epoch 76/1000
114/114          0s 3ms/step -
loss: 0.0119 - mae: 0.0119 - val_loss: 0.0087 - val_mae: 0.0087
Epoch 77/1000

```

```

114/114          0s 3ms/step -
loss: 0.0122 - mae: 0.0122 - val_loss: 0.0095 - val_mae: 0.0095
Epoch 78/1000
114/114          0s 3ms/step -
loss: 0.0121 - mae: 0.0121 - val_loss: 0.0085 - val_mae: 0.0085
Epoch 79/1000
114/114          0s 3ms/step -
loss: 0.0118 - mae: 0.0118 - val_loss: 0.0127 - val_mae: 0.0127
Epoch 80/1000
114/114          0s 3ms/step -
loss: 0.0117 - mae: 0.0117 - val_loss: 0.0079 - val_mae: 0.0079
Epoch 81/1000
114/114          0s 3ms/step -
loss: 0.0115 - mae: 0.0115 - val_loss: 0.0193 - val_mae: 0.0193
Epoch 82/1000
114/114          0s 3ms/step -
loss: 0.0118 - mae: 0.0118 - val_loss: 0.0107 - val_mae: 0.0107
Epoch 83/1000
114/114          0s 3ms/step -
loss: 0.0113 - mae: 0.0113 - val_loss: 0.0199 - val_mae: 0.0199
Epoch 84/1000
114/114          0s 3ms/step -
loss: 0.0118 - mae: 0.0118 - val_loss: 0.0108 - val_mae: 0.0108
Epoch 85/1000
114/114          0s 3ms/step -
loss: 0.0115 - mae: 0.0115 - val_loss: 0.0086 - val_mae: 0.0086
Epoch 86/1000
114/114          0s 3ms/step -
loss: 0.0116 - mae: 0.0116 - val_loss: 0.0092 - val_mae: 0.0092
Epoch 87/1000
114/114          0s 3ms/step -
loss: 0.0111 - mae: 0.0111 - val_loss: 0.0131 - val_mae: 0.0131
Epoch 88/1000
114/114          0s 3ms/step -
loss: 0.0110 - mae: 0.0110 - val_loss: 0.0094 - val_mae: 0.0094
Epoch 89/1000
114/114          0s 3ms/step -
loss: 0.0113 - mae: 0.0113 - val_loss: 0.0092 - val_mae: 0.0092
Epoch 90/1000
114/114          0s 3ms/step -
loss: 0.0112 - mae: 0.0112 - val_loss: 0.0135 - val_mae: 0.0135
Epoch 91/1000
114/114          0s 3ms/step -
loss: 0.0110 - mae: 0.0110 - val_loss: 0.0222 - val_mae: 0.0222
Epoch 92/1000
114/114          0s 3ms/step -
loss: 0.0111 - mae: 0.0111 - val_loss: 0.0160 - val_mae: 0.0160
Epoch 93/1000

```

```

114/114          0s 3ms/step -
loss: 0.0112 - mae: 0.0112 - val_loss: 0.0099 - val_mae: 0.0099
Epoch 94/1000
114/114          0s 3ms/step -
loss: 0.0109 - mae: 0.0109 - val_loss: 0.0075 - val_mae: 0.0075
Epoch 95/1000
114/114          0s 3ms/step -
loss: 0.0108 - mae: 0.0108 - val_loss: 0.0126 - val_mae: 0.0126
Epoch 96/1000
114/114          0s 3ms/step -
loss: 0.0108 - mae: 0.0108 - val_loss: 0.0139 - val_mae: 0.0139
Epoch 97/1000
114/114          0s 3ms/step -
loss: 0.0108 - mae: 0.0108 - val_loss: 0.0114 - val_mae: 0.0114
Epoch 98/1000
114/114          0s 3ms/step -
loss: 0.0108 - mae: 0.0108 - val_loss: 0.0129 - val_mae: 0.0129
Epoch 99/1000
114/114          0s 3ms/step -
loss: 0.0106 - mae: 0.0106 - val_loss: 0.0203 - val_mae: 0.0203
Epoch 100/1000
114/114          0s 3ms/step -
loss: 0.0106 - mae: 0.0106 - val_loss: 0.0073 - val_mae: 0.0073
Epoch 101/1000
114/114          0s 3ms/step -
loss: 0.0105 - mae: 0.0105 - val_loss: 0.0113 - val_mae: 0.0113
Epoch 102/1000
114/114          0s 3ms/step -
loss: 0.0105 - mae: 0.0105 - val_loss: 0.0068 - val_mae: 0.0068
Epoch 103/1000
114/114          0s 3ms/step -
loss: 0.0105 - mae: 0.0105 - val_loss: 0.0089 - val_mae: 0.0089
Epoch 104/1000
114/114          0s 3ms/step -
loss: 0.0103 - mae: 0.0103 - val_loss: 0.0159 - val_mae: 0.0159
Epoch 105/1000
114/114          0s 3ms/step -
loss: 0.0106 - mae: 0.0106 - val_loss: 0.0098 - val_mae: 0.0098
Epoch 106/1000
114/114          0s 3ms/step -
loss: 0.0104 - mae: 0.0104 - val_loss: 0.0135 - val_mae: 0.0135
Epoch 107/1000
114/114          0s 3ms/step -
loss: 0.0105 - mae: 0.0105 - val_loss: 0.0164 - val_mae: 0.0164
Epoch 108/1000
114/114          0s 3ms/step -
loss: 0.0106 - mae: 0.0106 - val_loss: 0.0228 - val_mae: 0.0228
Epoch 109/1000

```

```

114/114          0s 3ms/step -
loss: 0.0105 - mae: 0.0105 - val_loss: 0.0077 - val_mae: 0.0077
Epoch 110/1000
114/114          0s 3ms/step -
loss: 0.0108 - mae: 0.0108 - val_loss: 0.0081 - val_mae: 0.0081
Epoch 111/1000
114/114          0s 3ms/step -
loss: 0.0102 - mae: 0.0102 - val_loss: 0.0077 - val_mae: 0.0077
Epoch 112/1000
114/114          0s 3ms/step -
loss: 0.0103 - mae: 0.0103 - val_loss: 0.0095 - val_mae: 0.0095
Epoch 113/1000
114/114          0s 3ms/step -
loss: 0.0105 - mae: 0.0105 - val_loss: 0.0077 - val_mae: 0.0077
Epoch 114/1000
114/114          0s 3ms/step -
loss: 0.0103 - mae: 0.0103 - val_loss: 0.0112 - val_mae: 0.0112
Epoch 115/1000
114/114          0s 3ms/step -
loss: 0.0102 - mae: 0.0102 - val_loss: 0.0069 - val_mae: 0.0069
Epoch 116/1000
114/114          0s 3ms/step -
loss: 0.0103 - mae: 0.0103 - val_loss: 0.0099 - val_mae: 0.0099
Epoch 117/1000
114/114          0s 3ms/step -
loss: 0.0101 - mae: 0.0101 - val_loss: 0.0085 - val_mae: 0.0085
Epoch 118/1000
114/114          0s 3ms/step -
loss: 0.0100 - mae: 0.0100 - val_loss: 0.0164 - val_mae: 0.0164
Epoch 119/1000
114/114          0s 3ms/step -
loss: 0.0104 - mae: 0.0104 - val_loss: 0.0079 - val_mae: 0.0079
Epoch 120/1000
114/114          0s 3ms/step -
loss: 0.0101 - mae: 0.0101 - val_loss: 0.0071 - val_mae: 0.0071
Epoch 121/1000
114/114          0s 3ms/step -
loss: 0.0097 - mae: 0.0097 - val_loss: 0.0097 - val_mae: 0.0097
Epoch 122/1000
114/114          0s 3ms/step -
loss: 0.0100 - mae: 0.0100 - val_loss: 0.0150 - val_mae: 0.0150
Epoch 123/1000
114/114          0s 3ms/step -
loss: 0.0099 - mae: 0.0099 - val_loss: 0.0066 - val_mae: 0.0066
Epoch 124/1000
114/114          0s 3ms/step -
loss: 0.0098 - mae: 0.0098 - val_loss: 0.0119 - val_mae: 0.0119
Epoch 125/1000

```

```

114/114          0s 3ms/step -
loss: 0.0101 - mae: 0.0101 - val_loss: 0.0074 - val_mae: 0.0074
Epoch 126/1000
114/114          0s 3ms/step -
loss: 0.0096 - mae: 0.0096 - val_loss: 0.0098 - val_mae: 0.0098
Epoch 127/1000
114/114          0s 3ms/step -
loss: 0.0098 - mae: 0.0098 - val_loss: 0.0118 - val_mae: 0.0118
Epoch 128/1000
114/114          0s 3ms/step -
loss: 0.0101 - mae: 0.0101 - val_loss: 0.0080 - val_mae: 0.0080
Epoch 129/1000
114/114          0s 3ms/step -
loss: 0.0096 - mae: 0.0096 - val_loss: 0.0077 - val_mae: 0.0077
Epoch 130/1000
114/114          0s 3ms/step -
loss: 0.0099 - mae: 0.0099 - val_loss: 0.0101 - val_mae: 0.0101
Epoch 131/1000
114/114          0s 3ms/step -
loss: 0.0100 - mae: 0.0100 - val_loss: 0.0077 - val_mae: 0.0077
Epoch 132/1000
114/114          0s 3ms/step -
loss: 0.0099 - mae: 0.0099 - val_loss: 0.0147 - val_mae: 0.0147
Epoch 133/1000
114/114          0s 3ms/step -
loss: 0.0099 - mae: 0.0099 - val_loss: 0.0072 - val_mae: 0.0072
Epoch 134/1000
114/114          0s 3ms/step -
loss: 0.0099 - mae: 0.0099 - val_loss: 0.0067 - val_mae: 0.0067
Epoch 135/1000
114/114          0s 3ms/step -
loss: 0.0096 - mae: 0.0096 - val_loss: 0.0071 - val_mae: 0.0071
Epoch 136/1000
114/114          0s 3ms/step -
loss: 0.0098 - mae: 0.0098 - val_loss: 0.0111 - val_mae: 0.0111
Epoch 137/1000
114/114          0s 3ms/step -
loss: 0.0097 - mae: 0.0097 - val_loss: 0.0109 - val_mae: 0.0109
Epoch 138/1000
114/114          0s 3ms/step -
loss: 0.0096 - mae: 0.0096 - val_loss: 0.0138 - val_mae: 0.0138
Epoch 139/1000
114/114          0s 3ms/step -
loss: 0.0100 - mae: 0.0100 - val_loss: 0.0132 - val_mae: 0.0132
Epoch 140/1000
114/114          0s 3ms/step -
loss: 0.0096 - mae: 0.0096 - val_loss: 0.0117 - val_mae: 0.0117
Epoch 141/1000

```

```

114/114          0s 3ms/step -
loss: 0.0097 - mae: 0.0097 - val_loss: 0.0202 - val_mae: 0.0202
Epoch 142/1000
114/114          0s 3ms/step -
loss: 0.0098 - mae: 0.0098 - val_loss: 0.0068 - val_mae: 0.0068
Epoch 143/1000
114/114          0s 3ms/step -
loss: 0.0095 - mae: 0.0095 - val_loss: 0.0068 - val_mae: 0.0068
Epoch 144/1000
114/114          0s 3ms/step -
loss: 0.0093 - mae: 0.0093 - val_loss: 0.0136 - val_mae: 0.0136
Epoch 145/1000
114/114          0s 3ms/step -
loss: 0.0095 - mae: 0.0095 - val_loss: 0.0064 - val_mae: 0.0064
Epoch 146/1000
114/114          0s 3ms/step -
loss: 0.0093 - mae: 0.0093 - val_loss: 0.0146 - val_mae: 0.0146
Epoch 147/1000
114/114          0s 3ms/step -
loss: 0.0095 - mae: 0.0095 - val_loss: 0.0073 - val_mae: 0.0073
Epoch 148/1000
114/114          0s 3ms/step -
loss: 0.0093 - mae: 0.0093 - val_loss: 0.0087 - val_mae: 0.0087
Epoch 149/1000
114/114          0s 3ms/step -
loss: 0.0093 - mae: 0.0093 - val_loss: 0.0088 - val_mae: 0.0088
Epoch 150/1000
114/114          0s 3ms/step -
loss: 0.0094 - mae: 0.0094 - val_loss: 0.0094 - val_mae: 0.0094
Epoch 151/1000
114/114          0s 3ms/step -
loss: 0.0093 - mae: 0.0093 - val_loss: 0.0087 - val_mae: 0.0087
Epoch 152/1000
114/114          0s 3ms/step -
loss: 0.0094 - mae: 0.0094 - val_loss: 0.0070 - val_mae: 0.0070
Epoch 153/1000
114/114          0s 3ms/step -
loss: 0.0093 - mae: 0.0093 - val_loss: 0.0098 - val_mae: 0.0098
Epoch 154/1000
114/114          0s 3ms/step -
loss: 0.0093 - mae: 0.0093 - val_loss: 0.0099 - val_mae: 0.0099
Epoch 155/1000
114/114          0s 3ms/step -
loss: 0.0091 - mae: 0.0091 - val_loss: 0.0069 - val_mae: 0.0069
Epoch 156/1000
114/114          0s 3ms/step -
loss: 0.0093 - mae: 0.0093 - val_loss: 0.0146 - val_mae: 0.0146
Epoch 157/1000

```

```

114/114          0s 3ms/step -
loss: 0.0091 - mae: 0.0091 - val_loss: 0.0094 - val_mae: 0.0094
Epoch 158/1000
114/114          0s 3ms/step -
loss: 0.0092 - mae: 0.0092 - val_loss: 0.0136 - val_mae: 0.0136
Epoch 159/1000
114/114          0s 3ms/step -
loss: 0.0091 - mae: 0.0091 - val_loss: 0.0076 - val_mae: 0.0076
Epoch 160/1000
114/114          0s 3ms/step -
loss: 0.0092 - mae: 0.0092 - val_loss: 0.0061 - val_mae: 0.0061
Epoch 161/1000
114/114          0s 3ms/step -
loss: 0.0092 - mae: 0.0092 - val_loss: 0.0076 - val_mae: 0.0076
Epoch 162/1000
114/114          0s 3ms/step -
loss: 0.0088 - mae: 0.0088 - val_loss: 0.0067 - val_mae: 0.0067
Epoch 163/1000
114/114          0s 3ms/step -
loss: 0.0093 - mae: 0.0093 - val_loss: 0.0181 - val_mae: 0.0181
Epoch 164/1000
114/114          0s 3ms/step -
loss: 0.0092 - mae: 0.0092 - val_loss: 0.0065 - val_mae: 0.0065
Epoch 165/1000
114/114          0s 3ms/step -
loss: 0.0087 - mae: 0.0087 - val_loss: 0.0066 - val_mae: 0.0066
Epoch 166/1000
114/114          0s 3ms/step -
loss: 0.0088 - mae: 0.0088 - val_loss: 0.0091 - val_mae: 0.0091
Epoch 167/1000
114/114          0s 3ms/step -
loss: 0.0089 - mae: 0.0089 - val_loss: 0.0119 - val_mae: 0.0119
Epoch 168/1000
114/114          0s 3ms/step -
loss: 0.0088 - mae: 0.0088 - val_loss: 0.0156 - val_mae: 0.0156
Epoch 169/1000
114/114          0s 3ms/step -
loss: 0.0088 - mae: 0.0088 - val_loss: 0.0068 - val_mae: 0.0068
Epoch 170/1000
114/114          0s 3ms/step -
loss: 0.0090 - mae: 0.0090 - val_loss: 0.0108 - val_mae: 0.0108
Epoch 171/1000
114/114          0s 3ms/step -
loss: 0.0090 - mae: 0.0090 - val_loss: 0.0152 - val_mae: 0.0152
Epoch 172/1000
114/114          0s 3ms/step -
loss: 0.0091 - mae: 0.0091 - val_loss: 0.0079 - val_mae: 0.0079
Epoch 173/1000

```



```

114/114          0s 3ms/step -
loss: 0.0091 - mae: 0.0091 - val_loss: 0.0086 - val_mae: 0.0086
Epoch 174/1000
114/114          0s 3ms/step -
loss: 0.0087 - mae: 0.0087 - val_loss: 0.0114 - val_mae: 0.0114
Epoch 175/1000
114/114          0s 3ms/step -
loss: 0.0090 - mae: 0.0090 - val_loss: 0.0097 - val_mae: 0.0097
Epoch 176/1000
114/114          0s 3ms/step -
loss: 0.0087 - mae: 0.0087 - val_loss: 0.0087 - val_mae: 0.0087
Epoch 177/1000
114/114          0s 3ms/step -
loss: 0.0087 - mae: 0.0087 - val_loss: 0.0064 - val_mae: 0.0064
Epoch 178/1000
114/114          0s 3ms/step -
loss: 0.0088 - mae: 0.0088 - val_loss: 0.0075 - val_mae: 0.0075
Epoch 179/1000
114/114          0s 3ms/step -
loss: 0.0087 - mae: 0.0087 - val_loss: 0.0091 - val_mae: 0.0091
Epoch 180/1000
114/114          0s 3ms/step -
loss: 0.0085 - mae: 0.0085 - val_loss: 0.0070 - val_mae: 0.0070
Epoch 181/1000
114/114          0s 3ms/step -
loss: 0.0085 - mae: 0.0085 - val_loss: 0.0131 - val_mae: 0.0131
Epoch 182/1000
114/114          0s 3ms/step -
loss: 0.0087 - mae: 0.0087 - val_loss: 0.0091 - val_mae: 0.0091
Epoch 183/1000
114/114          0s 3ms/step -
loss: 0.0085 - mae: 0.0085 - val_loss: 0.0062 - val_mae: 0.0062
Epoch 184/1000
114/114          0s 3ms/step -
loss: 0.0085 - mae: 0.0085 - val_loss: 0.0118 - val_mae: 0.0118
Epoch 185/1000
114/114          0s 3ms/step -
loss: 0.0086 - mae: 0.0086 - val_loss: 0.0066 - val_mae: 0.0066
Epoch 186/1000
114/114          0s 3ms/step -
loss: 0.0085 - mae: 0.0085 - val_loss: 0.0121 - val_mae: 0.0121
Epoch 187/1000
114/114          0s 3ms/step -
loss: 0.0085 - mae: 0.0085 - val_loss: 0.0065 - val_mae: 0.0065
Epoch 188/1000
114/114          0s 3ms/step -
loss: 0.0086 - mae: 0.0086 - val_loss: 0.0102 - val_mae: 0.0102
Epoch 189/1000

```

```

114/114          0s 3ms/step -
loss: 0.0086 - mae: 0.0086 - val_loss: 0.0106 - val_mae: 0.0106
Epoch 190/1000
114/114          0s 3ms/step -
loss: 0.0084 - mae: 0.0084 - val_loss: 0.0147 - val_mae: 0.0147
Epoch 191/1000
114/114          0s 3ms/step -
loss: 0.0084 - mae: 0.0084 - val_loss: 0.0068 - val_mae: 0.0068
Epoch 192/1000
114/114          0s 3ms/step -
loss: 0.0082 - mae: 0.0082 - val_loss: 0.0064 - val_mae: 0.0064
Epoch 193/1000
114/114          0s 3ms/step -
loss: 0.0083 - mae: 0.0083 - val_loss: 0.0077 - val_mae: 0.0077
Epoch 194/1000
114/114          0s 3ms/step -
loss: 0.0083 - mae: 0.0083 - val_loss: 0.0069 - val_mae: 0.0069
Epoch 195/1000
114/114          0s 3ms/step -
loss: 0.0083 - mae: 0.0083 - val_loss: 0.0117 - val_mae: 0.0117
Epoch 196/1000
114/114          0s 3ms/step -
loss: 0.0082 - mae: 0.0082 - val_loss: 0.0068 - val_mae: 0.0068
Epoch 197/1000
114/114          0s 3ms/step -
loss: 0.0084 - mae: 0.0084 - val_loss: 0.0065 - val_mae: 0.0065
Epoch 198/1000
114/114          0s 3ms/step -
loss: 0.0080 - mae: 0.0080 - val_loss: 0.0111 - val_mae: 0.0111
Epoch 199/1000
114/114          0s 3ms/step -
loss: 0.0083 - mae: 0.0083 - val_loss: 0.0086 - val_mae: 0.0086
Epoch 200/1000
114/114          0s 3ms/step -
loss: 0.0086 - mae: 0.0086 - val_loss: 0.0154 - val_mae: 0.0154
Epoch 201/1000
114/114          0s 3ms/step -
loss: 0.0083 - mae: 0.0083 - val_loss: 0.0089 - val_mae: 0.0089
Epoch 202/1000
114/114          0s 3ms/step -
loss: 0.0083 - mae: 0.0083 - val_loss: 0.0103 - val_mae: 0.0103
Epoch 203/1000
114/114          0s 3ms/step -
loss: 0.0083 - mae: 0.0083 - val_loss: 0.0063 - val_mae: 0.0063
Epoch 204/1000
114/114          0s 3ms/step -
loss: 0.0082 - mae: 0.0082 - val_loss: 0.0099 - val_mae: 0.0099
Epoch 205/1000

```

```

114/114          0s 3ms/step -
loss: 0.0083 - mae: 0.0083 - val_loss: 0.0140 - val_mae: 0.0140
Epoch 206/1000
114/114          0s 3ms/step -
loss: 0.0081 - mae: 0.0081 - val_loss: 0.0142 - val_mae: 0.0142
Epoch 207/1000
114/114          0s 3ms/step -
loss: 0.0084 - mae: 0.0084 - val_loss: 0.0127 - val_mae: 0.0127
Epoch 208/1000
114/114          0s 3ms/step -
loss: 0.0081 - mae: 0.0081 - val_loss: 0.0066 - val_mae: 0.0066
Epoch 209/1000
114/114          0s 3ms/step -
loss: 0.0081 - mae: 0.0081 - val_loss: 0.0069 - val_mae: 0.0069
Epoch 210/1000
114/114          0s 3ms/step -
loss: 0.0082 - mae: 0.0082 - val_loss: 0.0079 - val_mae: 0.0079
Epoch 210: early stopping
Restoring model weights from the end of the best epoch: 160.

```

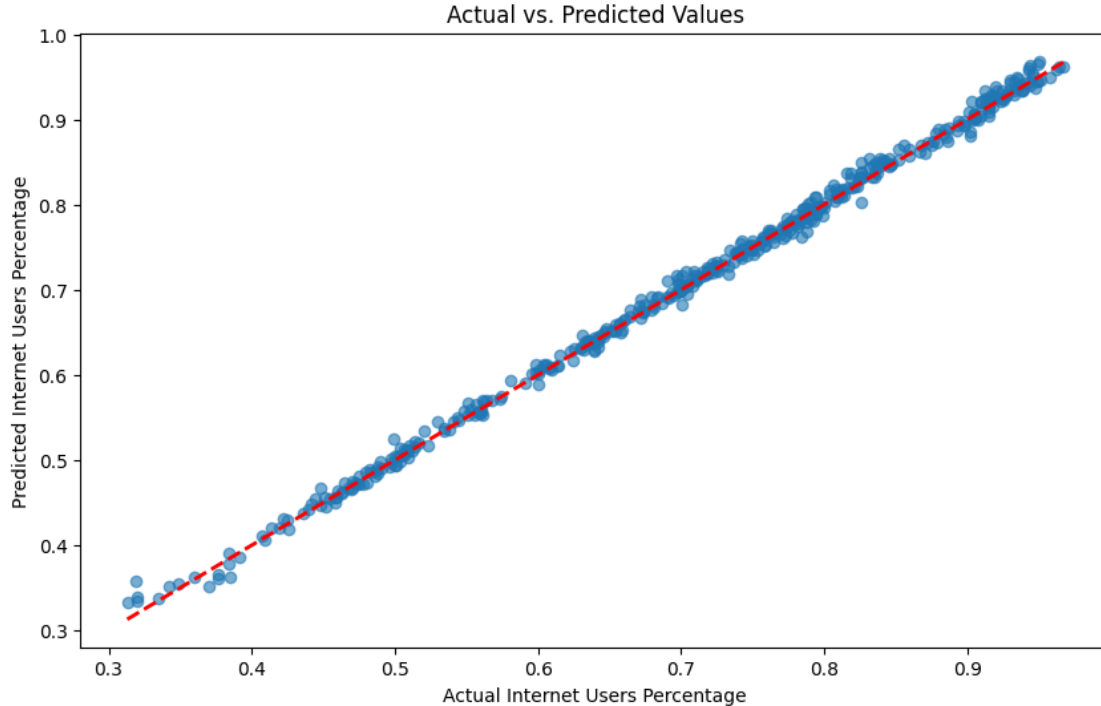
Final Model Loss on Test Data: 0.0061

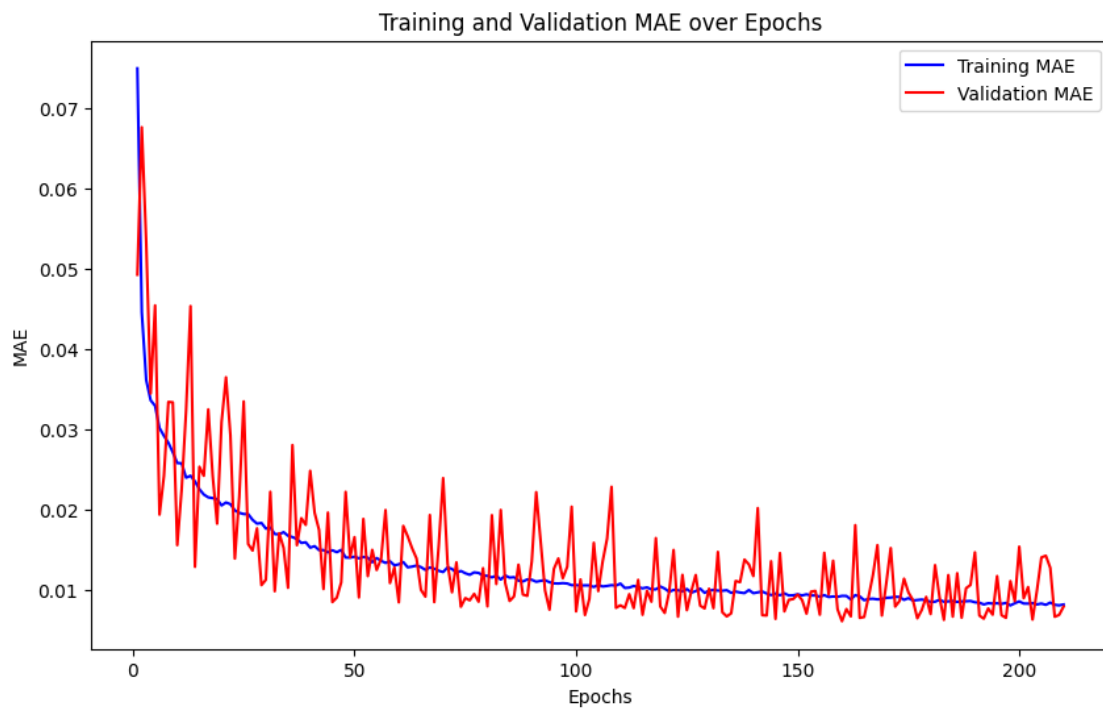
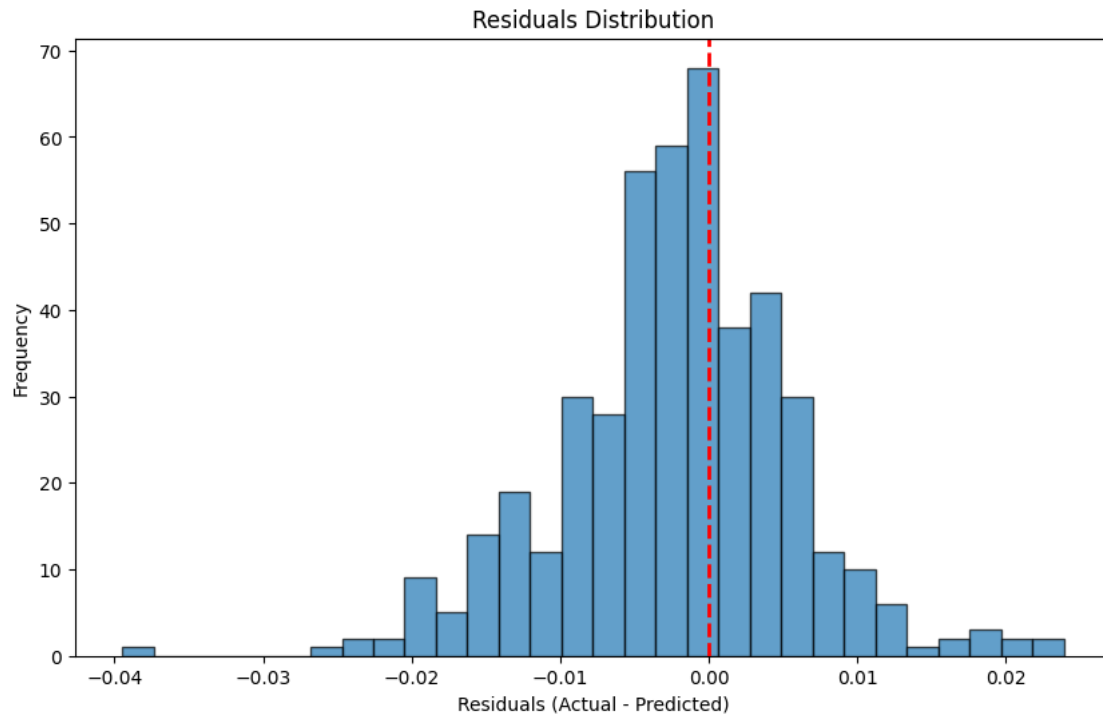
Final Model MAE on Test Data: 0.0061

```

15/15          0s 5ms/step

```





[]: